

Manual de Operação

OTS Manager CLI & Batch

Gerenciamento de identidades, grupos e QR Codes via API REST no OpenTAKServer

Controle do documento

Campo	Informação
Documento	Manual de Operação — OpenTAKServer Identity & Group Manager
Versão	1.0.1
Autor	Orlando Nascimento Santos
E-mail	onascimento@gmail.com
Plataforma	OpenTAKServer (OTS)
Interface	Python, CLI e API REST
Idioma	Português do Brasil

Objetivo. Este manual apresenta a instalação, configuração, operação, provisionamento em lote, geração de QR Codes, diagnóstico e boas práticas do utilitário `ots_manager.py`.

Sumário

1. Visão geral
2. Arquitetura e fluxo operacional
3. Pré-requisitos e instalação
4. Configuração por variáveis de ambiente
5. Segurança e proteção de credenciais
6. Mapeamento da API REST
7. Comandos da CLI
8. Provisionamento em lote
9. QR Codes e validade de acesso
10. Procedimentos operacionais
11. Tratamento de erros e diagnóstico
12. Boas práticas e limitações
13. Checklist de operação
14. Referência rápida
15. Apêndice: estrutura do código

Novidades da versão 1.0.1

- **Direções individuais por grupo:** `IN`, `OUT` ou `BOTH`, usando `GRUPO:DIREÇÃO` na CLI ou objetos JSON com `name` e `direction`.
- **Listagem de grupos:** o comando `list-groups` consulta e exibe os grupos existentes no OpenTAKServer.
- **Listagem de usuários:** o comando `list-users` exibe `username`, status de administrador e `last_login`, quando disponíveis.
- **Expansão de grupos:** `ALL`, `ALL:IN` e `ALL:OUT` associam o usuário a todos os grupos retornados pelo servidor.
- **Compatibilidade retroativa:** strings simples de grupo continuam válidas e usam `BOTH` por padrão.
- **Payload otimizado:** `exp` e `max` só são enviados quando informados.
- **Conformidade de segurança:** operações protegidas enviam `Referer`, `Origin` e tokens CSRF.

1. Visão geral

O **OTS Manager** é um utilitário Python para automatizar a administração do OpenTAKServer. Ele encapsula as operações necessárias para:

- autenticar no backend por meio de `/api/login`;
- manter uma sessão HTTP com cookies;
- recuperar e enviar tokens CSRF exigidos por Flask-WTF/Flask-Security;
- enviar os cabeçalhos `Referer` e `Origin`;
- criar grupos;
- criar usuários com confirmação obrigatória de senha;
- associar usuários a grupos nas direções `IN` e `OUT`;
- gerar strings e imagens PNG de QR Code para Android e iPhone;
- processar vários usuários a partir de um arquivo JSON;
- consolidar o resultado do processamento em um relatório JSON.

1.1 Modelo de operação

O fluxo recomendado é:

1. configurar `OTS_URL`, `OTS_USER` e `OTS_PASS`;
2. validar conectividade com o servidor;
3. executar o login;
4. criar os grupos necessários;
5. criar o usuário;
6. associar o usuário aos grupos em `IN` e `OUT`;
7. gerar o QR Code para o ecossistema correto;
8. armazenar o PNG e registrar o resultado;
9. validar a ativação no dispositivo ATAK/iTAK.

1.2 Ecossistemas suportados

Aplicativo	Endpoint	Operação	Resultado
Android / ATAK	<code>/api/atak_qr_string</code>	<code>POST</code>	String de configuração para QR Code
iPhone / iTAK	<code>/api/itak_qr_string</code>	<code>GET</code>	String de configuração para iOS

2. Arquitetura e fluxo operacional

O script utiliza `requests.Session()` para preservar cookies e cabeçalhos entre as chamadas. Após o login, o token CSRF pode ser obtido dos cookies `csrf_token`, `csrf_access_token` ou `XSRF-TOKEN`. Caso o servidor retorne o token no corpo da resposta, o utilitário também tenta recuperá-lo do JSON.

2.1 Fluxo de autenticação

```
ots_manager.py
|
| POST /api/login
v
OpenTAKServer
|
| cookies de sessão + token CSRF
v
Sessão requests.Session()
|
+--> Referer / Origin
+--> X-CSRFToken / X-CSRF-TOKEN
+--> operações protegidas
```

2.2 Cabeçalhos aplicados

Os cabeçalhos globais esperados são:

```
Content-Type: application/json
Referer: <OTS_URL>/
Origin: <OTS_URL>
```

Para operações protegidas, o token é enviado adicionalmente como:

```
X-CSRFToken: <token>
X-CSRF-TOKEN: <token>
```

3. Pré-requisitos e instalação

3.1 Requisitos

- Python 3.9 ou superior;
- acesso de rede ao OpenTAKServer;
- credencial de uma conta autorizada a criar usuários e grupos;
- permissões de escrita no diretório de trabalho;
- `pip` disponível no ambiente Python.

3.2 Instalação das dependências

```
python3 -m venv .venv
. .venv/bin/activate
python -m pip install --upgrade pip
pip install requests qrcode pillow
```

As bibliotecas têm as seguintes responsabilidades:

Biblioteca	Função
requests	Comunicação HTTP, sessão e cookies
qrcode	Construção da imagem do QR Code
Pillow	Backend de imagem utilizado pelo qrcode

3.3 Organização sugerida

```
ots-manager/
├── ots_manager.py
├── usuarios.json
├── qrcodes/
├── resultado_qr_codes.json
└── .venv/
```

Não versionar arquivos que contenham senhas, tokens, QR strings ou dados pessoais reais.

4. Configuração por variáveis de ambiente

Defina os parâmetros antes da execução:

```
export OTS_URL="http://opentakserver.example.com"
export OTS_USER="admin"
export OTS_PASS="sua_senha_aqui"
```

Nota — execução local sem Nginx: se o OpenTAKServer for executado localmente, diretamente pela API e sem passar pelo Nginx, informe a porta da API no `OTS_URL`. Por padrão, essa porta é **8081** (por exemplo, `http://localhost:8081`). Quando o acesso ocorrer pelo Nginx, use a URL publicada pelo proxy, normalmente sem informar a porta.

O script remove a barra final de `OTS_URL`. Na ausência das variáveis, os valores padrão são:

Variável	Padrão do código	Recomendação
OTS_URL	http://localhost	Sempre informar explicitamente
OTS_USER	admin	Usar conta nominal ou de serviço

Variável	Padrão do código	Recomendação
OTS_PASS	admin_password	Nunca depender do padrão

4.1 Teste de conectividade

```
curl -i "$OTS_URL/"
```

Se o serviço exigir HTTPS, use uma URL `https://` e valide o certificado. Evite desabilitar a validação TLS em produção.

4.2 Execução sem expor a senha no histórico

Quando possível, informe a senha por mecanismo seguro do ambiente. No mínimo, evite colocar a senha diretamente em comandos compartilhados ou scripts versionados:

```
read -r -s OTS_PASS
export OTS_PASS
python ots_manager.py --help
```

5. Segurança e proteção de credenciais

Atenção: o exemplo original utiliza HTTP e um endereço IP. Em produção, prefira HTTPS com certificado válido, rede restrita e uma conta com o menor privilégio necessário.

Boas práticas:

- não gravar `OTS_PASS` no repositório;
- não compartilhar `resultado_qr_codes.json` sem remover as QR strings;
- proteger os arquivos PNG gerados, pois eles podem permitir o provisionamento do dispositivo;
- usar contas de serviço com rotação de senha;
- restringir o acesso ao servidor por firewall ou VPN;
- registrar operações sem registrar senhas ou tokens;
- validar o destino antes de executar comandos destrutivos ou de provisionamento;
- remover arquivos temporários que contenham credenciais após o uso.

6. Mapeamento da API REST

Operação	Endpoint	Método	Dados principais	CSRF/Referer
Autenticação	<code>/api/login</code>	POST	<code>username</code> , <code>password</code>	Sessão inicial
Criar grupo	<code>/api/groups</code>	POST	<code>name</code>	Sim
Criar usuário	<code>/api/user/add</code>	POST	<code>username</code> , <code>password</code> , <code>confirm_password</code> , <code>email</code> , <code>administrator</code>	Conforme configuração do OTS

Operação	Endpoint	Método	Dados principais	CSRF/Referer
Vincular grupo	/api/users/groups	PUT	username, groups[], direction	Conforme configuração do OTS
QR Android	/api/atak_qr_string	POST	username, exp, nbf, max	Sim
QR iPhone	/api/itak_qr_string	GET	definido pelo servidor	Sessão autenticada

6.1 Campos obrigatórios e opcionais

A tabela abaixo consolida os campos aceitos pelo `ots_manager.py`. Campos indicados como **opcionais** podem ser omitidos; nesse caso, o script utiliza o valor padrão da CLI ou deixa o OpenTAKServer aplicar sua própria política.

Operação	Campos obrigatórios	Campos opcionais e comportamento padrão
Autenticação — /api/login	username, password	Nenhum. São obtidos de <code>OTS_USER</code> e <code>OTS_PASS</code> ; o código possui valores padrão, mas recomenda-se configurar ambos explicitamente.
Criar grupo — /api/groups	name	Nenhum.
Criar usuário — /api/user/add	username, password, confirm_password	email não é obrigatório e pode ser omitido ou ficar vazio; administrator é opcional e assume <code>false</code> . O confirm_password é preenchido automaticamente pelo script com o mesmo valor de password.
Vincular grupo — /api/users/groups	username, groups[] com strings ou objetos de direção, direction	Na CLI, direction é opcional e assume <code>BOTH</code> , executando as associações <code>IN</code> e <code>OUT</code> .
QR Android — /api/atak_qr_string	username	exp e max são opcionais . nbf é calculado automaticamente somente quando exp é informado. Se exp e max forem omitidos ou <code>null</code> , não são enviados e prevalece a política padrão do servidor.
QR iPhone — /api/itak_qr_string	Sessão autenticada	Não há campos informados na requisição <code>GET</code> ; a configuração é retornada pelo servidor.
CLI create-user	--username, --password	--email, --groups, --admin, --app, --exp, --max e --save-qr são opcionais. --app assume <code>android</code> ; --admin assume falso.
CLI qr	--username	--app, --exp, --max e --save-qr são opcionais. --app assume <code>android</code> .
CLI create-group	--name	Nenhum.
CLI link	--username, --group	--direction é opcional e assume <code>BOTH</code> .
CLI batch	--file	--output é opcional e assume <code>resultado_qr_codes.json</code> . Dentro de cada registro, username e password são obrigatórios; email, administrator, groups, app, expiration e max_uses são opcionais.

Resumo dos campos de usuário no lote

Campo JSON	Obrigatório?	Observação
<code>username</code>	Sim	Identificador do usuário no OTS.
<code>password</code>	Sim	Senha inicial; o script também envia <code>confirm_password</code> com o mesmo valor.
<code>email</code>	Não	Pode ser omitido, definido como <code>null</code> ou como string vazia, conforme a validação da instalação do OTS. O script o envia como vazio quando não informado.
<code>administrator</code>	Não	Assume <code>false</code> quando omitido.
<code>groups</code>	Não	Lista de grupos; quando informada, cada grupo é associado conforme sua direção (<code>IN</code> , <code>OUT</code> ou <code>BOTH</code>).
<code>app</code>	Não	Assume <code>android</code> ; também aceita <code>iphone</code> .
<code>expiration</code>	Não	Dias ou data <code>YYYY-MM-DD</code> ; quando ausente ou <code>null</code> , não há expiração enviada.
<code>max_uses</code>	Não	Limite de ativações; quando ausente ou <code>null</code> , não há limite enviado.

Importante: `email`, `expiration` e `max_uses` não são campos obrigatórios. A ausência desses campos não impede a criação do usuário nem a geração do QR Code; nesses casos, o servidor aplica os valores e políticas padrão. Já `username` e `password` são indispensáveis para cada usuário. O campo `confirm_password` é obrigatório para a API, mas é gerado automaticamente pelo script e não precisa ser informado separadamente na CLI ou no arquivo JSON.

6.2 Respostas consideradas sucesso

O utilitário trata como sucesso, conforme a operação, os códigos `200`, `201` e `204`. Um grupo ou usuário já existente pode ser tratado como situação idempotente quando a resposta `400` contém indicação de duplicidade, como `exists`.

6.2 Direções de grupo

A associação é feita separadamente:

- `IN`: mensagens recebidas pelo usuário;
- `OUT`: mensagens enviadas pelo usuário;
- `BOTH`: atalho da CLI que executa duas requisições, uma para cada direção.

Para tráfego tático bidirecional, a recomendação é sempre validar `IN` e `OUT`.

7. Comandos da CLI

7.1 Ajuda

```
python ots_manager.py --help
python ots_manager.py create-user --help
python ots_manager.py qr --help
```

7.2 Criar usuário e gerar QR Code Android

```
python ots_manager.py create-user \
-u "piloto1" \
-p "Senha123!" \
-g CSAR \
--app android
```

O comando cria o grupo, cria o usuário, associa o grupo nas direções **IN** e **OUT**, solicita a string ao OTS e grava um PNG com nome padrão semelhante a `piloto1_android.png`.

7.3 Criar usuário e gerar QR Code iPhone

```
python ots_manager.py create-user \
-u "piloto2" \
-p "Senha123!" \
-g CSAR \
--app iphone
```

7.4 Informar e-mail e perfil administrativo

```
python ots_manager.py create-user \
-u "operador" \
-p "SenhaForte!" \
-e "operador@empresa.com" \
--admin \
--app android
```

Use `--admin` somente quando a função operacional realmente exigir privilégios administrativos.

7.5 Validade e limite de ativações

```
python ots_manager.py create-user \  
-u "convidado" \  
-p "Senha123!" \  
-g Visitantes \  
--app android \  
--exp 30 \  
--max 1
```

`--exp 30` representa 30 dias a partir do momento da geração. Também é possível usar uma data no formato `YYYY-MM-DD`:

```
python ots_manager.py qr -u "piloto1" --app android --exp 2026-12-31 --max  
2
```

Quando `--exp` e `--max` são omitidos, esses campos não são enviados e o servidor aplica suas políticas padrão.

7.6 Gerar QR para usuário existente

```
python ots_manager.py qr -u "piloto1" --app android  
python ots_manager.py qr -u "piloto2" --app iphone --save-qr  
piloto2_ios.png
```

7.7 Criar grupo isolado

```
python ots_manager.py create-group -n "Patrulha"
```

7.8 Vincular usuário a grupo

```
python ots_manager.py link -u "piloto1" -g "Patrulha"
```

Direção específica:

```
python ots_manager.py link -u "piloto1" -g "Patrulha" --direction IN  
python ots_manager.py link -u "piloto1" -g "Patrulha" --direction OUT
```

7.9 Listar grupos e usuários

```
python ots_manager.py list-groups  
python ots_manager.py list-users
```

`list-groups` consulta os grupos existentes. `list-users` exhibe `username`, status de administrador e `last_login`, quando disponíveis.

7.10 Associar um usuário a todos os grupos

Use `ALL`, `ALL:IN` ou `ALL:OUT` no valor de `--group` ou `--groups`:

```
python ots_manager.py create-user -u "usuario_global" -p "Senha123!" -g ALL --app android
python ots_manager.py create-user -u "usuario_global_out" -p "Senha123!" -g ALL:OUT --app android
python ots_manager.py create-user -u "usuario_global_in" -p "Senha123!" -g ALL:IN --app iphone
python ots_manager.py link -u "usuario_global" -g ALL --direction OUT
```

`ALL` sem sufixo usa `BOTH`; `ALL:IN` usa somente `IN`; `ALL:OUT` usa somente `OUT`.

8. Provisionamento em lote

8.1 Arquivo `usuarios.json`

8.1.1 Grupos com direções individuais

Cada item de `groups` pode ser uma string simples ou um objeto com `name` e `direction`:

```
[
  {
    "username": "operador_alpha",
    "password": "SenhaSegura123!",
    "groups": [
      {"name": "Grupo_IN", "direction": "IN"},
      {"name": "Grupo_OUT", "direction": "OUT"},
      {"name": "Grupo_BIDIRECIONAL", "direction": "BOTH"}
    ],
    "app": "android"
  }
]
```

`parse_group_entry()` aceita `IN`, `OUT` e `BOTH`; uma string como `"CSAR"` equivale a um objeto com direção `BOTH`.

```
[
  {
    "username": "operador_alpha",
    "password": "SenhaForte123!",
    "email": "alpha@empresa.com",
    "administrator": false,
    "groups": ["CSAR", "Resgate"],
    "app": "android"
  },
  {
    "username": "operador_bravo",
    "password": "SenhaForte456!",
    "email": "bravo@empresa.com",
    "administrator": false,
    "groups": ["CSAR"],
    "app": "iphone"
  },
  {
    "username": "operador_temporario",
    "password": "SenhaTemporaria789!",
    "email": "temp@empresa.com",
    "administrator": false,
    "groups": ["Operacoes"],
    "app": "android",
    "expiration": 30,
    "max_uses": 1
  }
]
```

As chaves `expiration` e `max_uses` são opcionais. Se estiverem ausentes ou com valor `null`, as restrições não serão enviadas ao servidor.

8.2 Execução

```
python ots_manager.py batch \
  -f usuarios.json \
  -o resultado_qr_codes.json
```

O processamento:

1. cria todos os grupos encontrados;
2. processa cada usuário;
3. associa os grupos em `IN` e `OUT`;
4. converte a validade para Unix Epoch quando necessário;
5. solicita o QR Code;
6. grava PNGs em `qr_codes/`;
7. exporta o relatório consolidado.

8.4 Lote com todos os grupos e direção

```
[
  {"username": "global_um", "password": "SenhaGlobal1!", "groups":
["ALL:OUT"], "app": "android"},
  {"username": "global_dois", "password": "SenhaGlobal2!", "groups":
["ALL:IN"], "app": "iphone", "expiration": 14},
  {"username": "global_tres", "password": "SenhaGlobal3!", "groups":
["ALL"], "app": "android"}
]
```

```
python ots_manager.py batch -f usuarios_all.json -o resultado_all.json
```

ALL:OUT usa somente **OUT**; **ALL:IN** usa somente **IN**; **ALL** sem sufixo usa **BOTH**.

8.3 Estrutura do relatório

```
[
  {
    "username": "operador_alpha",
    "app": "android",
    "max_uses": "padrão do servidor",
    "expiration": "padrão do servidor",
    "qr_string": "string retornada pelo OTS",
    "qr_image": "qrcores/operador_alpha_android.png"
  }
]
```

O relatório deve ser tratado como material sensível. Em ambientes reais, considere gerar uma versão operacional sem a chave **qr_string** para compartilhamento.

9. QR Codes e validade de acesso

9.1 Android

O endpoint Android recebe **username**. Os campos opcionais são:

Campo	Significado
exp	instante de expiração em Unix Epoch
nbf	instante a partir do qual o QR é válido
max	limite de ativações

Quando existe expiração, o script calcula **nbf** como o instante atual em UTC.

9.2 iPhone

O endpoint iPhone é consultado por `GET` e retorna diretamente a string de configuração do iTAK. A resposta pode ser JSON, com chaves como `qr_string` ou `itak_qr_string`, ou texto simples.

9.3 Conversão de datas

A função `parse_expiration` aceita:

- número de dias, como `30`;
- data absoluta, como `2026-12-31`;
- valores vazios ou equivalentes a `none`, `null`, `eterno` e `infinito`, tratados como ausência de restrição.

Datas absolutas são interpretadas em UTC. Confirme o fuso e a política do servidor antes de usar datas de expiração em operações críticas.

10. Procedimentos operacionais

10.1 Provisionar uma nova equipe

1. confirmar o endereço do OTS e a conta de operação;
2. listar os grupos necessários;
3. revisar o JSON com outro operador;
4. executar o lote em janela autorizada;
5. conferir o relatório e os PNGs;
6. distribuir cada QR Code somente ao destinatário correto;
7. testar login e tráfego CoT no dispositivo;
8. registrar a execução e a data de validade.

10.2 Reemitir um QR Code

```
python ots_manager.py qr \  
  --username "usuario_existente" \  
  --app android \  
  --save-qr "reemitido_usuario_android.png"
```

Confirme se a reemissão invalida ou não o QR anterior de acordo com a política do OTS.

10.3 Validar conectividade pós-provisionamento

No dispositivo:

- importar o QR Code no aplicativo correspondente;
- confirmar o endereço do servidor;
- confirmar que o usuário consegue autenticar;
- testar recebimento de uma mensagem CoT;
- testar envio de uma mensagem CoT;
- validar se os grupos esperados estão aplicados.

11. Tratamento de erros e diagnóstico

Falha de conexão

Sintoma: erro de conexão ou timeout.

Verificações:

```
curl -i "$OTS_URL/"
getent hosts <hostname>
```

Confirme IP, porta, firewall, VPN e se o serviço do OTS está ativo.

Falha de autenticação

Sintoma: mensagem `Falha na autenticação` com status HTTP diferente de `200` ou `201`.

Verifique usuário, senha, URL, método de login e se a conta está habilitada.

HTTP 400 ao criar grupo ou usuário

Pode indicar dados inválidos ou duplicidade. O script reconhece duplicidade quando o corpo contém `exists`; caso contrário, leia a resposta completa e corrija os campos enviados.

HTTP 401 ou 403 em operação protegida

Verifique:

- se o login realmente criou a sessão;
- se o token CSRF existe nos cookies ou na resposta;
- se os cabeçalhos `X-CSRFToken` e `X-CSRF-TOKEN` foram enviados;
- se `Referer` e `Origin` correspondem ao `OTS_URL`;
- se a conta possui permissão para a operação;
- se a sessão expirou.

QR Code vazio ou inválido

Confirme o endpoint selecionado (`android` ou `iphone`), o usuário, o status HTTP e o formato da resposta. Preserve o corpo retornado durante o diagnóstico, sem expor tokens em logs públicos.

PNG não gerado

Confirme a instalação de `qrcode` e `pillow`, a existência do diretório de destino e as permissões de escrita:

```
python -c "import qrcode, PIL; print('dependências OK')"
```

Lote parcialmente concluído

O lote pode criar alguns recursos antes de falhar em um registro posterior. Não execute novamente às cegas. Compare o relatório, verifique quais usuários e grupos existem e trate duplicidades de forma controlada.

12. Boas práticas e limitações

- Faça backup seguro do arquivo de entrada antes de uma execução em lote.
- Use nomes de usuário estáveis e convenções documentadas.
- Evite espaços e caracteres ambíguos em identificadores.
- Teste primeiro com uma conta de laboratório.
- Não distribua QR Codes por canais públicos.
- Registre a data de geração, validade e responsável pela entrega.
- Valide sempre as duas direções de grupo quando houver tráfego bidirecional.
- O modo `BOTH` executa duas requisições; uma falha pode deixar a associação incompleta.
- O tratamento de “já existe” não substitui a conferência do estado atual no servidor.
- O endpoint e o formato das respostas podem variar conforme a versão do OpenTAKServer.
- O script original não implementa rollback transacional. Para lotes críticos, use etapas menores e reconciliação posterior.
- A senha aparece na linha de comando nos exemplos; em produção, prefira um mecanismo seguro de entrada.

12.1 Observação de manutenção do código

Ao transcrever ou atualizar o código-fonte, valide a indentação do bloco `get_qr_string`, especialmente os ramos `elif app_type == "iphone"` e `else`. Execute uma compilação sintática antes do uso:

```
python -m py_compile ots_manager.py
```

Também é recomendável adicionar testes para `parse_expiration`, seleção dos endpoints, tratamento de respostas JSON/texto e montagem dos payloads.

13. Checklist de operação

Antes da execução

- [] `OTS_URL` aponta para o ambiente correto.
- [] O acesso de rede foi validado.
- [] A conta tem permissão suficiente.
- [] A senha não está versionada.
- [] O JSON foi validado e revisado.
- [] Os nomes de grupos e usuários estão corretos.
- [] `app` é `android` ou `iphone`.
- [] Expiração e limite de uso foram conferidos.

Depois da execução

- [] O login foi bem-sucedido.

- ☐ Os grupos foram criados ou confirmados.
- ☐ Os usuários foram criados ou confirmados.
- ☐ Cada grupo foi associado em **IN** e **OUT**.
- ☐ O QR string foi retornado.
- ☐ O PNG foi gerado e aberto para conferência.
- ☐ O relatório JSON foi salvo em local protegido.
- ☐ O dispositivo foi testado nos dois sentidos de comunicação.

14. Referência rápida

```
# Instalação
pip install requests qrcode pillow

# Configuração
export OTS_URL="http://servidor-ots"
export OTS_USER="admin"
export OTS_PASS="..."

# Usuário Android
python ots_manager.py create-user -u piloto1 -p 'Senha123!' -g CSAR --app
android

# Usuário iPhone
python ots_manager.py create-user -u piloto2 -p 'Senha123!' -g CSAR --app
iphone

# QR existente
python ots_manager.py qr -u piloto1 --app android --save-qr piloto1.png

# Grupo
python ots_manager.py create-group -n Patrulha

# Associação bidirecional
python ots_manager.py link -u piloto1 -g Patrulha

# Lote
python ots_manager.py batch -f usuarios.json -o resultado_qr_codes.json

# Validação sintática
python -m py_compile ots_manager.py

# Listar grupos e usuários
python ots_manager.py list-groups
python ots_manager.py list-users

# Criar usuário em todos os grupos com direção específica
python ots_manager.py create-user -u usuario_global -p 'Senha123!' -g
ALL:OUT --app android
python ots_manager.py create-user -u usuario_global_in -p 'Senha123!' -g
ALL:IN --app iphone

# Associar usuário existente a todos os grupos
python ots_manager.py link -u usuario_global -g ALL --direction BOTH
```

15. Apêndice: estrutura do código

O arquivo `ots_manager.py` está organizado pelas funções abaixo:

Função	Responsabilidade
<code>get_csrf_token()</code>	Recupera o token CSRF da sessão
<code>login()</code>	Autentica e atualiza os cabeçalhos
<code>create_group()</code>	Cria ou confirma um grupo
<code>create_user()</code>	Cria ou confirma um usuário
<code>add_user_to_group()</code>	Associa grupos em <code>IN</code> , <code>OUT</code> ou <code>BOTH</code>
<code>parse_expiration()</code>	Converte dias/data para Unix Epoch
<code>get_qr_string()</code>	Obtém a configuração Android ou iPhone
<code>save_qr_code_image()</code>	Gera e salva a imagem PNG
<code>list_groups()</code>	Recupera a lista atual de grupos do servidor
<code>list_users()</code>	Lista usuários e resume administrador/último login
<code>list_groups()</code>	Recupera os grupos atuais do servidor
<code>list_users()</code>	Lista usuários, administrador e último login
<code>process_batch_list()</code>	Orquestra o lote e expande <code>ALL</code>
<code>main()</code>	Define a CLI e despacha os comandos

Encerramento

O OTS Manager reduz tarefas manuais e padroniza o provisionamento no OpenTAKServer. A automação deve ser utilizada junto com controle de acesso, proteção de credenciais, revisão dos arquivos de entrada e validação funcional no dispositivo ATAK/ITAK.

Manual de Operação OTS Manager — versão 1.0.1

Criado por Orlando Nascimento Santos — onascimento@gmail.com

Documento elaborado com base na especificação e no código-fonte fornecidos no manual original.

Nota: o código-fonte completo `ots_manager.py` é mantido separadamente no repositório. Este manual documenta sua operação, comandos, payloads e resultados.